

Complete AI Engineering Guide: From Fundamentals to Advanced LLM Concepts

Table of Contents

1. [Model Architectures](#)
 2. [NLP Basics](#)
 3. [Key LLM Concepts](#)
 4. [Evaluation and Optimization](#)
 5. [Security and Responsible AI](#)
 6. [Practical Implementation Examples](#)
-

Model Architectures

LSTM (Long Short-Term Memory) Networks

LSTM networks were revolutionary in handling sequential data and solving the vanishing gradient problem in traditional RNNs.

Key Components:

- **Forget Gate:** Decides what information to discard
- **Input Gate:** Determines what new information to store
- **Output Gate:** Controls what parts of the cell state to output

```
import torch
import torch.nn as nn
import numpy as np

class LSTMModel(nn.Module):
    def __init__(self, input_size, hidden_size, num_layers, output_size):
        super(LSTMModel, self).__init__()
        self.hidden_size = hidden_size
        self.num_layers = num_layers

        self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        # Initialize hidden and cell states
        h0 = torch.zeros(self.num_layers, x.size(0), self.hidden_size)
        c0 = torch.zeros(self.num_layers, x.size(0), self.hidden_size)

        # Forward propagate LSTM
        out, _ = self.lstm(x, (h0, c0))

        # Decode the hidden state of the last time step
```

```

        out = self.fc(out[:, -1, :])
        return out

# Example usage for sequence prediction
input_size = 10
hidden_size = 20
num_layers = 2
output_size = 1
seq_length = 30
batch_size = 32

model = LSTMModel(input_size, hidden_size, num_layers, output_size)
sample_input = torch.randn(batch_size, seq_length, input_size)
output = model(sample_input)
print(f"LSTM output shape: {output.shape}")

```

Sequence-to-Sequence (Seq2Seq) Models

Seq2Seq models consist of an encoder-decoder architecture, fundamental for tasks like translation and summarization.

```

class Seq2SeqEncoder(nn.Module):
    def __init__(self, vocab_size, embed_size, hidden_size, num_layers):
        super(Seq2SeqEncoder, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embed_size)
        self.lstm = nn.LSTM(embed_size, hidden_size, num_layers, batch_first=True)

    def forward(self, x):
        embedded = self.embedding(x)
        outputs, (hidden, cell) = self.lstm(embedded)
        return outputs, hidden, cell

class Seq2SeqDecoder(nn.Module):
    def __init__(self, vocab_size, embed_size, hidden_size, num_layers):
        super(Seq2SeqDecoder, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embed_size)
        self.lstm = nn.LSTM(embed_size, hidden_size, num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_size, vocab_size)

    def forward(self, x, hidden, cell):
        embedded = self.embedding(x)
        output, (hidden, cell) = self.lstm(embedded, (hidden, cell))
        prediction = self.fc(output)
        return prediction, hidden, cell

class Seq2Seq(nn.Module):
    def __init__(self, encoder, decoder):
        super(Seq2Seq, self).__init__()
        self.encoder = encoder
        self.decoder = decoder

```

```
def forward(self, source, target):
    # Encode the source sequence
    _, hidden, cell = self.encoder(source)

    # Decode using the encoded context
    outputs, _, _ = self.decoder(target, hidden, cell)
    return outputs
```

BERT (Bidirectional Encoder Representations from Transformers)

BERT revolutionized NLP by introducing bidirectional context understanding through masked language modeling.

```
from transformers import BertTokenizer, BertModel, BertForMaskedLM
import torch

class BERTClassifier(nn.Module):
    def __init__(self, bert_model_name, num_classes, dropout_rate=0.1):
        super(BERTClassifier, self).__init__()
        self.bert = BertModel.from_pretrained(bert_model_name)
        self.dropout = nn.Dropout(dropout_rate)
        self.classifier = nn.Linear(self.bert.config.hidden_size, num_classes)

    def forward(self, input_ids, attention_mask=None):
        outputs = self.bert(input_ids=input_ids, attention_mask=attention_mask)
        pooled_output = outputs.pooler_output
        pooled_output = self.dropout(pooled_output)
        logits = self.classifier(pooled_output)
        return logits

# Example usage
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BERTClassifier('bert-base-uncased', num_classes=2)

# Sample text processing
text = "This is a sample sentence for BERT processing."
inputs = tokenizer(text, return_tensors='pt', padding=True, truncation=True)
with torch.no_grad():
    outputs = model(**inputs)
    print(f"BERT classification output shape: {outputs.shape}")
```

GPT (Generative Pre-trained Transformer)

GPT models use the decoder-only transformer architecture for autoregressive language generation.

```
import torch.nn.functional as F

class GPTBlock(nn.Module):
    def __init__(self, embed_size, heads, dropout, forward_expansion):
```

```

    super(GPTBlock, self).__init__()
    self.attention = nn.MultiheadAttention(embed_size, heads, dropout=dropout)
    self.norm1 = nn.LayerNorm(embed_size)
    self.norm2 = nn.LayerNorm(embed_size)

    self.feed_forward = nn.Sequential(
        nn.Linear(embed_size, forward_expansion * embed_size),
        nn.ReLU(),
        nn.Linear(forward_expansion * embed_size, embed_size)
    )
    self.dropout = nn.Dropout(dropout)

def forward(self, x, mask=None):
    # Self-attention with residual connection
    attention_out, _ = self.attention(x, x, x, attn_mask=mask)
    x = self.norm1(attention_out + x)

    # Feed-forward with residual connection
    forward_out = self.feed_forward(x)
    out = self.norm2(forward_out + x)
    return self.dropout(out)

class SimpleGPT(nn.Module):
    def __init__(self, vocab_size, embed_size, num_layers, heads, max_length,
dropout=0.1):
        super(SimpleGPT, self).__init__()
        self.embed_size = embed_size
        self.token_embedding = nn.Embedding(vocab_size, embed_size)
        self.position_embedding = nn.Embedding(max_length, embed_size)

        self.layers = nn.ModuleList([
            GPTBlock(embed_size, heads, dropout, forward_expansion=4)
            for _ in range(num_layers)
        ])

        self.ln_f = nn.LayerNorm(embed_size)
        self.head = nn.Linear(embed_size, vocab_size, bias=False)

def forward(self, x):
    seq_len = x.size(1)
    positions = torch.arange(0, seq_len).unsqueeze(0)

    # Token + Position embeddings
    x = self.token_embedding(x) + self.position_embedding(positions)

    # Create causal mask for autoregressive generation
    mask = torch.triu(torch.ones(seq_len, seq_len) * float('-inf'),
diagonal=1)

    # Pass through transformer blocks
    for layer in self.layers:
        x = layer(x, mask)

    x = self.ln_f(x)

```

```
logits = self.head(x)
return logits
```

Transformer Architecture Deep Dive

The Transformer is the foundation of modern LLMs, using self-attention mechanisms to process sequences in parallel.

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super(MultiHeadAttention, self).__init__()
        assert d_model % num_heads == 0

        self.d_model = d_model
        self.num_heads = num_heads
        self.d_k = d_model // num_heads

        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)

    def scaled_dot_product_attention(self, Q, K, V, mask=None):
        attn_scores = torch.matmul(Q, K.transpose(-2, -1)) / np.sqrt(self.d_k)

        if mask is not None:
            attn_scores = attn_scores.masked_fill(mask == 0, -1e9)

        attn_probs = torch.softmax(attn_scores, dim=-1)
        output = torch.matmul(attn_probs, V)
        return output, attn_probs

    def forward(self, x, mask=None):
        batch_size, seq_len, d_model = x.size()

        # Linear transformations and reshape
        Q = self.W_q(x).view(batch_size, seq_len, self.num_heads,
self.d_k).transpose(1, 2)
        K = self.W_k(x).view(batch_size, seq_len, self.num_heads,
self.d_k).transpose(1, 2)
        V = self.W_v(x).view(batch_size, seq_len, self.num_heads,
self.d_k).transpose(1, 2)

        # Apply attention
        attn_output, attn_weights = self.scaled_dot_product_attention(Q, K, V,
mask)

        # Concatenate heads and put through final linear layer
        attn_output = attn_output.transpose(1, 2).contiguous().view(batch_size,
seq_len, d_model)
        output = self.W_o(attn_output)
```

```

        return output, attn_weights

class TransformerBlock(nn.Module):
    def __init__(self, d_model, num_heads, d_ff, dropout):
        super(TransformerBlock, self).__init__()
        self.attention = MultiHeadAttention(d_model, num_heads)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)

        self.feed_forward = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(d_ff, d_model)
        )
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, mask=None):
        # Multi-head attention with residual connection
        attn_output, _ = self.attention(x, mask)
        x = self.norm1(x + self.dropout(attn_output))

        # Feed-forward with residual connection
        ff_output = self.feed_forward(x)
        x = self.norm2(x + self.dropout(ff_output))

        return x

```

NLP Basics

Tokenization

Tokenization is the process of breaking down text into smaller units (tokens) that can be processed by machine learning models.

```

import re
from collections import Counter
from typing import List, Dict

class BasicTokenizer:
    def __init__(self):
        self.word_pattern = re.compile(r'\b\w+\b')

    def tokenize(self, text: str) -> List[str]:
        """Basic word tokenization"""
        return self.word_pattern.findall(text.lower())

    def sentence_tokenize(self, text: str) -> List[str]:
        """Simple sentence tokenization"""

```

```

    sentences = re.split(r'[.!?]+', text)
    return [s.strip() for s in sentences if s.strip()]

class SubwordTokenizer:
    def __init__(self, vocab_size: int = 10000):
        self.vocab_size = vocab_size
        self.vocab = {}
        self.merges = []

    def get_word_frequencies(self, texts: List[str]) -> Dict[str, int]:
        """Get frequency of words in corpus"""
        word_freqs = Counter()
        for text in texts:
            words = text.split()
            for word in words:
                word_freqs[' '.join(word) + ' </w>'] += 1
        return dict(word_freqs)

    def get_pairs(self, word_freqs: Dict[str, int]) -> Counter:
        """Get all pairs of consecutive symbols"""
        pairs = Counter()
        for word, freq in word_freqs.items():
            symbols = word.split()
            for i in range(len(symbols) - 1):
                pairs[(symbols[i], symbols[i + 1])] += freq
        return pairs

    def merge_vocab(self, pair: tuple, word_freqs: Dict[str, int]) -> Dict[str,
int]:
        """Merge the most frequent pair"""
        new_word_freqs = {}
        bigram = re.escape(' '.join(pair))
        p = re.compile(r'(?<!\S)' + bigram + r'(?!\S)')

        for word in word_freqs:
            new_word = p.sub(' '.join(pair), word)
            new_word_freqs[new_word] = word_freqs[word]
        return new_word_freqs

    def train_bpe(self, texts: List[str]):
        """Train Byte Pair Encoding"""
        word_freqs = self.get_word_frequencies(texts)

        # Initialize vocabulary with individual characters
        vocab = set()
        for word in word_freqs:
            for char in word.split():
                vocab.add(char)

        # Perform BPE merges
        for i in range(self.vocab_size - len(vocab)):
            pairs = self.get_pairs(word_freqs)
            if not pairs:
                break

```

```

        best_pair = pairs.most_common(1)[0][0]
        word_freqs = self.merge_vocab(best_pair, word_freqs)
        self.merges.append(best_pair)
        vocab.add(''.join(best_pair))

    self.vocab = {word: i for i, word in enumerate(sorted(vocab))}

# Example usage
tokenizer = BasicTokenizer()
text = "Natural Language Processing is fascinating! It involves many complex algorithms."
tokens = tokenizer.tokenize(text)
print(f"Tokens: {tokens}")

sentences = tokenizer.sentence_tokenize(text)
print(f"Sentences: {sentences}")

# BPE example
bpe_tokenizer = SubwordTokenizer(vocab_size=1000)
corpus = [
    "the quick brown fox jumps over the lazy dog",
    "the dog was lazy and slept all day",
    "quick brown foxes are clever animals"
]
bpe_tokenizer.train_bpe(corpus)
print(f"BPE vocab size: {len(bpe_tokenizer.vocab)}")

```

Vectorization and Embeddings

Converting text to numerical representations that machine learning models can process.

```

import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
from gensim.models import Word2Vec
import torch

class TextVectorizer:
    def __init__(self):
        self.count_vectorizer = CountVectorizer()
        self.tfidf_vectorizer = TfidfVectorizer()

    def bag_of_words(self, texts: List[str]) -> np.ndarray:
        """Convert texts to bag-of-words representation"""
        return self.count_vectorizer.fit_transform(texts).toarray()

    def tfidf(self, texts: List[str]) -> np.ndarray:
        """Convert texts to TF-IDF representation"""
        return self.tfidf_vectorizer.fit_transform(texts).toarray()

    def one_hot_encode(self, text: str, vocab: Dict[str, int]) -> np.ndarray:

```

```

        """Simple one-hot encoding for words"""
        vector = np.zeros(len(vocab))
        words = text.lower().split()
        for word in words:
            if word in vocab:
                vector[vocab[word]] = 1
        return vector

class Word2VecEmbeddings:
    def __init__(self, vector_size=100, window=5, min_count=1, workers=4):
        self.vector_size = vector_size
        self.window = window
        self.min_count = min_count
        self.workers = workers
        self.model = None

    def train(self, sentences: List[List[str]]):
        """Train Word2Vec model"""
        self.model = Word2Vec(
            sentences=sentences,
            vector_size=self.vector_size,
            window=self.window,
            min_count=self.min_count,
            workers=self.workers
        )

    def get_embedding(self, word: str) -> np.ndarray:
        """Get embedding for a word"""
        if self.model and word in self.model.wv:
            return self.model.wv[word]
        return np.zeros(self.vector_size)

    def similarity(self, word1: str, word2: str) -> float:
        """Calculate similarity between two words"""
        if self.model and word1 in self.model.wv and word2 in self.model.wv:
            return self.model.wv.similarity(word1, word2)
        return 0.0

class PositionalEncoding(nn.Module):
    """Positional encoding for transformer models"""
    def __init__(self, d_model: int, max_len: int = 5000):
        super(PositionalEncoding, self).__init__()

        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)

        div_term = torch.exp(torch.arange(0, d_model, 2).float() *
                               (-np.log(10000.0) / d_model))

        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        pe = pe.unsqueeze(0).transpose(0, 1)

        self.register_buffer('pe', pe)

```

```
def forward(self, x):
    return x + self.pe[:x.size(0), :]

# Example usage
texts = [
    "machine learning is powerful",
    "deep learning uses neural networks",
    "natural language processing is complex"
]

vectorizer = TextVectorizer()
bow_vectors = vectorizer.bag_of_words(texts)
tfidf_vectors = vectorizer.tfidf(texts)

print(f"Bag of Words shape: {bow_vectors.shape}")
print(f"TF-IDF shape: {tfidf_vectors.shape}")

# Word2Vec training
sentences = [text.split() for text in texts]
w2v = Word2VecEmbeddings()
w2v.train(sentences)

# Get embeddings
word_embedding = w2v.get_embedding("learning")
print(f"Word embedding shape: {word_embedding.shape}")

# Similarity
similarity = w2v.similarity("machine", "deep")
print(f"Similarity between 'machine' and 'deep': {similarity}")
```

Key LLM Concepts

Retrieval-Augmented Generation (RAG)

RAG combines retrieval systems with generation models to provide more accurate and up-to-date responses.

```
import faiss
from sentence_transformers import SentenceTransformer
from transformers import GPT2LMHeadModel, GPT2Tokenizer
import numpy as np

class RAGSystem:
    def __init__(self, model_name='sentence-transformers/all-MiniLM-L6-v2'):
        self.encoder = SentenceTransformer(model_name)
        self.knowledge_base = []
        self.embeddings = None
        self.index = None

        # Initialize generation model
```

```

self.generator_tokenizer = GPT2Tokenizer.from_pretrained('gpt2')
self.generator_model = GPT2LMHeadModel.from_pretrained('gpt2')
self.generator_tokenizer.pad_token = self.generator_tokenizer.eos_token

def add_documents(self, documents: List[str]):
    """Add documents to the knowledge base"""
    self.knowledge_base.extend(documents)

    # Encode documents
    doc_embeddings = self.encoder.encode(documents)

    if self.embeddings is None:
        self.embeddings = doc_embeddings
    else:
        self.embeddings = np.vstack([self.embeddings, doc_embeddings])

    # Build FAISS index
    dimension = self.embeddings.shape[1]
    self.index = faiss.IndexFlatIP(dimension) # Inner Product for similarity

    # Normalize embeddings for cosine similarity
    faiss.normalize_L2(self.embeddings)
    self.index.add(self.embeddings)

def retrieve(self, query: str, k: int = 3) -> List[str]:
    """Retrieve top-k relevant documents"""
    if self.index is None:
        return []

    query_embedding = self.encoder.encode([query])
    faiss.normalize_L2(query_embedding)

    scores, indices = self.index.search(query_embedding, k)

    retrieved_docs = []
    for i, score in zip(indices[0], scores[0]):
        if i < len(self.knowledge_base):
            retrieved_docs.append(self.knowledge_base[i])

    return retrieved_docs

def generate_response(self, query: str, context: str, max_length: int = 100):
    """Generate response based on query and retrieved context"""
    prompt = f"Context: {context}\n\nQuestion: {query}\n\nAnswer:"

    inputs = self.generator_tokenizer(prompt, return_tensors='pt',
                                      truncation=True, max_length=512)

    with torch.no_grad():
        outputs = self.generator_model.generate(
            inputs['input_ids'],
            max_length=inputs['input_ids'].shape[1] + max_length,
            num_return_sequences=1,
            temperature=0.7,

```

```

        pad_token_id=self.generator_tokenizer.eos_token_id
    )

    response = self.generator_tokenizer.decode(outputs[0],
skip_special_tokens=True)
    # Extract only the generated part
    response = response[len(prompt):].strip()

    return response

def query(self, question: str, k: int = 3) -> str:
    """Complete RAG pipeline"""
    # Retrieve relevant documents
    retrieved_docs = self.retrieve(question, k)

    if not retrieved_docs:
        return "No relevant information found."

    # Combine retrieved documents as context
    context = " ".join(retrieved_docs)

    # Generate response
    response = self.generate_response(question, context)

    return response

# Example usage
rag = RAGSystem()

# Add knowledge base
documents = [
    "Machine learning is a subset of artificial intelligence that focuses on algorithms.",
    "Deep learning uses neural networks with multiple layers to process data.",
    "Natural language processing deals with the interaction between computers and humans.",
    "Transformers are a type of neural network architecture used in NLP tasks.",
    "BERT is a bidirectional transformer model for language understanding."
]

rag.add_documents(documents)

# Query the system
question = "What is deep learning?"
answer = rag.query(question)
print(f"Question: {question}")
print(f"Answer: {answer}")

```

LLMOps (Large Language Model Operations)

LLMOps encompasses the practices for deploying, monitoring, and maintaining LLM systems in production.

```

import logging
import time
from datetime import datetime
from typing import Dict, Any, List
import json
import psutil
import threading

class LLMMonitor:
    def __init__(self):
        self.metrics = {
            'requests_count': 0,
            'total_latency': 0,
            'error_count': 0,
            'start_time': time.time()
        }
        self.request_history = []
        self.setup_logging()

    def setup_logging(self):
        """Setup logging for LLMOps monitoring"""
        logging.basicConfig(
            level=logging.INFO,
            format='%(asctime)s - %(levelname)s - %(message)s',
            handlers=[
                logging.FileHandler('llm_ops.log'),
                logging.StreamHandler()
            ]
        )
        self.logger = logging.getLogger(__name__)

    def log_request(self, request_data: Dict[str, Any], response_data: Dict[str, Any],
                    latency: float, error: str = None):
        """Log individual request metrics"""
        self.metrics['requests_count'] += 1
        self.metrics['total_latency'] += latency

        if error:
            self.metrics['error_count'] += 1

        request_log = {
            'timestamp': datetime.now().isoformat(),
            'request': request_data,
            'response': response_data,
            'latency': latency,
            'error': error,
            'memory_usage': psutil.virtual_memory().percent,
            'cpu_usage': psutil.cpu_percent()
        }

        self.request_history.append(request_log)
        self.logger.info(f"Request processed - Latency: {latency:.3f}s, Error:

```

```

{error}")

def get_metrics(self) -> Dict[str, float]:
    """Get current performance metrics"""
    uptime = time.time() - self.metrics['start_time']
    avg_latency = (self.metrics['total_latency'] /
                  max(1, self.metrics['requests_count']))
    error_rate = (self.metrics['error_count'] /
                 max(1, self.metrics['requests_count'])) * 100
    throughput = self.metrics['requests_count'] / max(1, uptime / 60) #
requests/minute

    return {
        'total_requests': self.metrics['requests_count'],
        'average_latency': avg_latency,
        'error_rate': error_rate,
        'throughput': throughput,
        'uptime_hours': uptime / 3600
    }

def health_check(self) -> Dict[str, Any]:
    """Perform system health check"""
    memory = psutil.virtual_memory()
    disk = psutil.disk_usage('/')

    health_status = {
        'status': 'healthy',
        'timestamp': datetime.now().isoformat(),
        'memory_usage': memory.percent,
        'disk_usage': (disk.used / disk.total) * 100,
        'cpu_usage': psutil.cpu_percent(interval=1),
        'metrics': self.get_metrics()
    }

    # Determine health status based on thresholds
    if (memory.percent > 90 or
        health_status['cpu_usage'] > 95 or
        health_status['metrics']['error_rate'] > 10):
        health_status['status'] = 'unhealthy'
    elif (memory.percent > 75 or
          health_status['cpu_usage'] > 80 or
          health_status['metrics']['error_rate'] > 5):
        health_status['status'] = 'warning'

    return health_status

class ModelVersionManager:
    def __init__(self):
        self.models = {}
        self.active_model = None
        self.logger = logging.getLogger(__name__)

    def register_model(self, model_id: str, model, metadata: Dict[str, Any]):
        """Register a new model version"""

```

```

        self.models[model_id] = {
            'model': model,
            'metadata': metadata,
            'registered_at': datetime.now().isoformat(),
            'usage_count': 0
        }
        self.logger.info(f"Model {model_id} registered with metadata: {metadata}")

def set_active_model(self, model_id: str):
    """Set the active model for inference"""
    if model_id in self.models:
        self.active_model = model_id
        self.logger.info(f"Active model set to: {model_id}")
    else:
        raise ValueError(f"Model {model_id} not found")

def get_active_model(self):
    """Get the currently active model"""
    if self.active_model:
        self.models[self.active_model]['usage_count'] += 1
        return self.models[self.active_model]['model']
    return None

def compare_models(self) -> Dict[str, Any]:
    """Compare registered models"""
    comparison = {}
    for model_id, model_info in self.models.items():
        comparison[model_id] = {
            'metadata': model_info['metadata'],
            'usage_count': model_info['usage_count'],
            'registered_at': model_info['registered_at']
        }
    return comparison

class LLMInferenceService:
    def __init__(self):
        self.monitor = LLMMonitor()
        self.version_manager = ModelVersionManager()
        self.rate_limiter = {}
        self.max_requests_per_minute = 100

    def rate_limit_check(self, user_id: str) -> bool:
        """Simple rate limiting implementation"""
        current_time = time.time()
        if user_id not in self.rate_limiter:
            self.rate_limiter[user_id] = []

        # Remove old requests (older than 1 minute)
        self.rate_limiter[user_id] = [
            req_time for req_time in self.rate_lim

```